uc3m | Universidad Carlos III de Madrid

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Computational algorithms for pricing financial derivatives under local volatility models

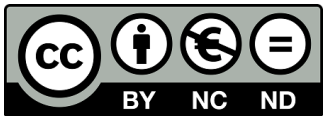
---

Jose Enrique Zafra Mena

Jorge Morón Vidal
Francisco Manuel Bernal Martínez

Madrid, 2026

AVOID PLAGIARISM
The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



SUMMARY

Keywords:

DEDICATION

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1. Motivation

1.2. Main Objectives

1.3. Key contributions

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

2.2. Stochastic Calculus Background

TODO: add Cumulative Distribution Function definition
TODO: add Probability Density Function definition (and delete from below)

[Oorsterlee book] The *Characteristic Function* (ChF) for $u \in \mathbb{R}$ of the random variable X is the Fourier-Stieltjes transform of the cumulative distribution function $F_X(x)$.

$$\phi_X(u) := \mathbb{E}\left[e^{iuX}\right] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \tag{2.1}$$

where $f_X(u)$ is the *Probability Density Function*, given by

$$f_X(u) = \frac{\partial F_X(u)}{\partial u}. \tag{2.2}$$

Applying the logarithm to the ChF, we define the *Cumulant Characteristic Function* by

$$\zeta_X(u) = \log \mathbb{E}\left[e^{iux}\right] = \log \phi_X(u). \tag{2.3}$$

If we compute the MacLaurin expansion to ζ_X, we arrive to

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_k(u)}{\partial u^k}\bigg|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \tag{2.4}$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k}\Big|_{u=0}$ is the k-th *cumulant*.

Furthermore, we can define the *Moment Generating Function*, $\mathcal{M}_X(u)$, by

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}\left[e^{ux}\right]. \tag{2.5}$$

Since the MGF is $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, we can compute the cumulants, using the ChF, as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu)\bigg|_{u=0}. \tag{2.6}$$

2.3. The Heston Stochastic Volatility Model

2.4. Pricing Via Characteristic Functions

2.5. The Fourier-COS method

TODO: rellenar

The COS method is the ...
The main reasons we want to use a cosine expansion are:

- In a cosine Fourier expansion we can represent even functions around 0 exactly.
- We can extend any function $g : [0, \pi] \to \mathbb{R}$ to become an even function on $[-\pi, \pi]$ as follows:
$$\bar{g}(\theta) = \begin{cases} g(\theta), & \theta \geq 0, \\ g(-\theta), & \theta < 0. \end{cases} \tag{2.7}$$
- For functions supported on any other finite interval, say $[a, b] \in \mathbb{R}$, the Fourier cos series expansion can be obtained via a change of variables:
$$\theta := \frac{y-a}{b-a}\pi, \quad y := \frac{b-a}{\pi}\theta + a.$$

TODO: falta más teoría para enlazar

We finally arrive to the approximation of the PDF

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \tag{2.8}$$

where $\sum'$ indicates that the first term should be multiplied by $1/2$, and the coefficients depend on the ChF:

$$\bar{F}_k := \frac{2}{b-a} \operatorname{Re}\left[\phi_X\left(\frac{\pi k}{b-a}\right) \cdot \exp\left(-\frac{i\pi a k}{b-a}\right)\right]. \tag{2.9}$$

TODO: no se si poner algo aquí o nueva subsection

The value of a plain vanilla option is given by

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x) dy, \tag{2.10}$$

where $\tau = T - t_0$, $f_X(T, y; t_0, x)$ is the transition probability density of $X(t)$, and r the interest rate.

Applying the COS method to the last expression, we arrive to

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1}{}' \ \mathrm{Re}\left[\phi_X\left(\frac{\pi k}{b-a}\right) \cdot \exp\left(-\frac{i\pi a k}{b-a}\right)\right] \cdot H_k, \tag{2.11}$$

with x a function of $S(t_0)$, $\phi_X(u)$ the ChF, and H_K the *payoff coefficients*:

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos\left(\frac{y-a}{b-a}\pi k\right) dy. \tag{2.12}$$

TODO: seguir con los payoffs coeffients

2.5.1. The COS Method Applied To The Heston Model

TODO: rellenar

We need to define the following functions:

$$D_1(u) = \sqrt{(\kappa - \gamma\rho i u)^2 + (u^2 + iu)\,\gamma^2}, \tag{2.13}$$

$$g(u) = \frac{\kappa - \gamma\rho i u - D_1}{\kappa - \gamma\rho i u + D_1}. \tag{2.14}$$

Using a vector of strikes $\mathbf{K}$, we find the COS formula for the Heston model:

$$V(t_0, \mathbf{x}) \approx \mathbf{K} e^{-r\tau} \cdot \mathrm{Re}\left\{\sum_{k=0}^{N-1}{}' \ \varphi_H\left[\frac{\pi k}{b-a}, T; v(t_0)\right] U_k \cdot \exp\left[i\pi k \frac{\mathbf{X}(t_0) - a}{b-a}\right]\right\}, \tag{2.15}$$

where no se si es la ChF?

$$\begin{aligned} \varphi_H(t, T; v(t_0)) = \exp&\left\{iur\tau + \frac{v(t_0)}{\gamma^2}\left(\frac{1 - e^{-D_1\tau}}{1 - ge^{-D_1\tau}}\right)(\kappa - i\rho\gamma u - D_1)\right\} \cdot \\ &\cdot \exp\left\{\frac{\kappa\bar{v}}{\gamma^2}\left(\tau(\kappa - i\rho\gamma u - D_1) - 2\log\left(\frac{1 - ge^{-D_1\tau}}{1 - g}\right)\right)\right\}. \end{aligned} \tag{2.16}$$

2.6. Implied Volatility And Numerical Inversion

2.7. State Of The Art

3. NEURAL NETWORK PRICER FOR HESTON VANILLA OPTIONS

Nuestro objetivo en este apartado es construir un surrogate pricer para opciones vanilla bajo el modelo de Heston con volatilidad estocástica usando redes neuronales.

El pricing clásico, como es COS (FFT, PDE, etc) es muy preciso, pero es costoso. Sin embargo, la red neuronal aproxima el mapa directo $(\Theta; m, \tau, r) \rightarrow \sigma_{imp}$. Es importante notar que en este capitulo nos centramos en la red de pricing, no de calibración.

The objective of the network is to learn a smooth and accurate approximation of the implied volatility as a function of the model parameters and contract characteristics.

3.1. Problem Formulation

3.2. Synthetic Data Formulation

We need data to train the neural network. Para tener una fuente controlada y reproducible, vamos a trabajar con datos sintéticos, generados a partir del método de COS, para alimentar a la red neuronal.

Los datos que la red necesita son vectores de tamaño n, donde hay $n-1$ parámetros en las primeras columnas, y en la n-ésima tenemos la volatilidad implícita, calculada con el método de COS y Brent usando los $n-1$ parámetros.

Esto conlleva un problema principal, y es que vamos a trabajar en alta dimensionalidad: tenemos el set de los parámetros de Heston, $\Theta = \{\rho, \kappa, v_0, \bar{v}, \gamma\}$, y el set de los parámetros dados por la opción y el mercado, $\Sigma = \{K, S_0, \tau, r\}$. Esto quiere decir que tenemos que generar un espacio $\mathbb{R}^9$ de 9 dimensiones.

Si quisieramos crear un grid "clásico" para poblar este espacio, tomando N puntos de cada parámetro, necesitaríamos N^9 puntos, Solamente con tomar $N = 100$, ya se vuelve un problema extremadamente costoso.

Para reducir un poco este problema, si fijamos el strike price en $K = 1$, podemos trabajar en unidades de *moneyness*, $m = S_0/K$, De esta forma, ahora trabajamos en $\mathbb{R}^8$.TODO: quizas esto ponerlo en 3.2.1.

Aún así, el problema de poblar este espacio sigue siendo muy costoso. Por eso, uti-

lizaremos un método de sampleo muy útil en este tipo de problemas, conocido como el *Latin Hypercube Sampling* (LHS).

3.2.1. Parameter Ranges And Constraints

3.2.2. Sampling Strategy: Latin Hypercube Sampling

The training of the neural network requires generating a large set of model parameters distributed over a high-dimensional domain. In our case, the parameter space is $\mathbb{R}^8$. A purely random sampling of this space may generate highly populated regions while leaving other regions scarcely explored. As a consequence, the resulting training dataset may be poorly representative, leading to suboptimal training performance.

To avoid this problem, we will use the *Latin Hypercube Sampling* (LHS), a sampling method designed to uniformly cover each dimension. The main idea consists in splitting each dimension of the parameter space into N disjoint subintervals of equal length, and randomly selecting exactly one sample within each subinterval.

Thus, LHS guarantees a one-dimensional marginal distribution along each coordinate, independently of the total dimension of the space. This results in a sampling scheme that is more representative of the parameter space.

This creates a representative distribution sampling of the parameter space. In contrast to a full-grid sampling, the total number of samples N remains fixed, preventing exponential growth in the number of dimensions.

poner referencia?

3.2.3. Dataset Splitting And Preprocessing

3.3. Neural Network Architecture

3.3.1. Model Class

Following [1], the pricer is implemented as a fully connected feed-forward neural network (FCNN). This class of models is particularly well suited for option pricing problems where the input consists of a low-dimensional vector of continuous variables and no explicit spatial or temporal structure is present.

The neural network approximates the nonlinear mapping

$$f_{NN} : \mathbb{R}^8 \to \mathbb{R},$$

where the input vector is given by

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{\nu}, \nu_0, m, \tau, r),$$

and the scalar output corresponds to the implied volatility of a European vanilla option.

The network is used as a surrogate pricer, replacing the numerical evaluation of the Heston model combined with the implied volatility inversion.

3.3.2. Architecture Details

3.3.3. Output Interpretation

3.4. Training Procedure

TODO: rellenar

3.4.1. Loss Function

3.4.2. Optimizer And Hyperparameters

3.4.3. Regularization And Callbacks

4. CALIBRATION NEURAL NETWORK FOR THE HESTON MODEL

4.1. Calibration problem formulation

4.2. Input representation: implied volatility surfaces

4.3. Synthetic calibration dataset

4.4. CaNN architecture

5. PINN FOR GREEKS COMPUTATION

6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

6.1. Pricing Accuracy

6.2. Implied Volatility Accuracy

6.3. Computational Efficiency

6.4. Generalization And Robustness

7. CONCLUSION AND FUTURE WORK

7.1. Market Data

7.2. Exotic Derivatives

7.3. Greeks And Hedging

7.4. Model Extension

7.5. Machine Learning Improvements

BIBLIOGRAPHY

[1] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. DOI: 10.1186/s13362-019-0066-7. Accessed: Oct. 21, 2025. [Online]. Available: https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7.

APPENDIX A: ANALYSIS OF THE COS METHOD ERROR

no se si dejarlo como apéndice o meterlo en alguna seccion

For the COS method, we have three sources of error:

- The integration range truncation error, ε_1.
- The series truncation error on $[a, b]$, ε_2.
- The error related to approximating $\bar{A}_k(x)$ by $\bar{F}_k(x)$, ε_3 (poner ref a alguna ecuacion).

It is important to note that there is no error in the payoff coefficient H_k for plain vanilla European options.

If the integration range $[a, b]$ is chosen sufficiently large, the overall error is dominated by ε_2 (TODO: argumentarlo). Also, ε_2 converges exponentially if $f_X(x) \in C^\infty[a, b]$ (TODO: justificarlo tamb). Otherwise, converges only algebraically.

To choose an optimal integration range we will use the expression (TODO: ref a Oorsterlee)

$$[a, b] = \left[(x + \zeta_1) - L\sqrt{\zeta_2 + \sqrt{\zeta_4}},\ (x + \zeta_1) + L\sqrt{\zeta_2 + \sqrt{\zeta_4}} \right], \tag{7.1}$$

where $L \in [6, 12]$ is a parameter that depends on the tolerance we want to have, and ζ_i are the cumulants, computed using the expression (2.6).

APPENDIX B: CALIBRATION OF THE HESTON MODEL

TODO: